03 — Field validation

What it detects: internally inconsistent documents — the kind of arithmetic or date mistake that falsified or hand-edited documents often contain. It is **pure rule logic over one document's own extracted fields**: no I/O, no model, no database.

Code: `services/field_validation_service.py` (`validate_fields`), wrapped by `tasks/document_checks.py` (`run_document_checks`), which stores a `document_checks` row with `check_type = field_validation` . Highlight regions come from `services/field_regions.py` and `services/field_exception_service.py` .

Sub-checks

`validate_fields(extracted_fields) -> {"result": "pass"|"flag", "details": {...}}` runs these independent sub-checks; the overall result is `flag` if **any** sub-check is `flag` .

Sub-check	Flags when	Skipped when
`date_in_future`	the document's `date` core field is after today	no parseable date
`line_item_arithmetic`	a line's quantity × unit price (− its discount) ≠ the line total printed	no line with quantity, unit price and total
`subtotal_line_item_consistency`	the line items that count toward the total don't add up to the `subtotal` — or, with no subtotal shown, to the `amount` (less `tax_amount` when tax is shown). Not summed: rows that are totals themselves ("Total Fees", "Grand total", "Subtotal", "Amount due"), and installment rows ("Term I/II/III", "Installment 1/2", "Payment 1") that split another line — their sum equals that line (CASE-346FA23E: Term I + II + III = Tuition Fees, so the payable items are 38,834 against a total of 34,012)	no line totals, or no subtotal/total
(`subtotal_line_item_consistency` , multi-column tables)	when the line items come from a table with separate amount columns (Price / Discount / Net / VAT / Total — `line_item_parsing.parse_multi_column_tables` , which keeps each row's values per column and reads OCR's "D.DD" as 0.00 and "131,900,00" as 131,900.00), **each column is summed on its own** and compared with its own total row: Net with "Total Net Price" (else the subtotal), VAT with "VAT" (else the tax amount), Total with "Total Price" (else the total); a Net column with no Total column beside it is compared with the total. Never one row's Net Price with another's Total Price (CASE-D87E2E82: 120,007.50 / 6,000.37 / 126,007.87, all matching). When only the VAT column can be compared, the plain check runs instead	—
`installment_consistency`	two or more installment rows come within 20 % of another line (the largest, at least as big as each installment) without adding up to it — a part of the schedule was changed. Passes, with the sum spelled out, when they add up exactly	no installment rows splitting another line (terms that are the charges themselves are not installments)
`date_sequence`	the dates are out of order: the page was printed (browser print header, first line: "8/18/23, 12:54 PM"; month/day first, day/month when that is impossible) more than a day before its own date, or the PDF was created more than a day before the page was printed. Passes with the sequence ("dated 17 Aug 2023 → printed 18 Aug 2023 → PDF created 21 Aug 2023: in order")	no print header

Sub-check	Flags when	Skipped when
`fake_scan_watermark`	a scanner app's line ("Scanned with CamScanner", "Adobe Scan", Microsoft Lens…) on a file with **no raster image** (`pdf_info.image_count` = 0): nothing was scanned, the line was typed in	no watermark; image count not recorded (documents extracted before it was)
`stamp_authenticity`	a detected stamp is live text and/or vector lines in the file (`material` recorded by signature/stamp detection — `services/forensics/pdf_facts.py`), not an image of an ink stamp. `synthetic_stamp_unsigned` : such a stamp and no signature anywhere	no stamp; detection not finished; material not recorded
`tax_invoice_trn`	titled TAX INVOICE / فاتورة ضريبية but no TRN shown: no labelled number (TRN, Tax Registration, VAT No, الرقم الضريبي; Western or Arabic-Indic digits) and no UAE TRN (100…) anywhere. Whether a shown TRN is valid is `iban_trn_validation` 's question	not titled a tax invoice
`multiple_invoices`	the file holds several invoices (`services/multi_invoice.py` : a new invoice number starts a new invoice; each invoice's pages — text layer first, OCR for scanned pages — extracted on their own into `extracted_fields.invoices`) and two are dated in the same month	one invoice in the file
`per_invoice_checks`	an invoice of a multi-invoice file fails its own line-item / subtotal / total-and-tax / tax-rate check. A kind of problem the document's own sub-check already flags is listed but not flagged again	one invoice in the file
`total_tax_consistency`	`amount` ≠ `subtotal + tax_amount` (or ≠ `subtotal` when no tax line is shown)	no subtotal
`tax_rate_consistency`	`tax_amount` ≠ taxable base × `tax_rate` (base = subtotal, or total less tax)	no rate or no tax amount
`amount_in_words_consistency`	the amount written in words ≠ the total (or the subtotal) in figures	no amount in words, or it can't be read
`reference_number_format`	the reference number is not a plausible identifier (regex `^[A-Za-z0-9](?:[A-Za-z0-9\-/_. #]*[A-Za-z0-9])?$`)	—
`stamp_issuer_consistency`	the text read on the document's stamp does not name the issuer (rapidfuzz `token_set_ratio` < 80, per writing system, with addresses, P.O. boxes, branch numbers, legal forms and UAE place names left out, abbreviations expanded — MBZ = Mohamed Bin Zayed, MBR = Mohammed Bin Rashid, Intl = International, Pvt = Private — and generic words dropped: School, Campus, LLC, Branch, Private)	detection not finished; no stamp text read; no issuer; stamp and issuer share no script

Sub-check	Flags when	Skipped when
`document_date_vs_file_creation`	the printed `date` is more than 1 day after the PDF's own CreationDate **and** the file was modified on or after that date (an older file edited and re-dated). A file created before its date and never modified since — generated in advance — passes	no date; no CreationDate in `pdf_info` (documents extracted before it was recorded); a CreationDate at or before 2000-01-01 (unset clock)

Calculation checks

Extraction returns structured `line_items` (description, quantity, unit price, discount, line total, and whether the line counts toward the total) and `amount_in_words` (the words and the number they spell), stored in `documents.extracted_fields`. The model is told to copy every number **exactly as printed and never recompute** one: the discrepancies are the point. On a real scanned invoice it returned `3 × 6,000 = 16,600` as printed, and all three errors on that page were flagged.

- **Amounts are compared to the cent** (one cent of rounding per term, plus half a cent per unit when a unit price is multiplied out), not by a percentage. A 1 % allowance on a 55,500 total would hide a 400 discrepancy.
- **Excluded lines:** a line the document marks as already paid or informational doesn't count toward the total. If including it is what makes the sum match, the check passes rather than flagging the model's judgment.
- **Amount in words:** English words are read by code (`services/amount_words.py`), not taken from the model, which tends to "correct" words that contradict the figures. Other languages (Arabic) use the model's reading. Words that leave out the fils/cents still match.
- Each wrong line total and the amount-in-words line are located on the page (`services/field_locator_service.py`) and drawn as field-exception boxes.

Line items are also read deterministically from the OCR layout where the model is unreliable (`services/line_item_parsing.py`): one-row fee tables, `<rate> PER <unit> X <qty> = <total>` lines, and "Description | Amount" tables whose rows OCR stacked into one cell (`12,000 9,000 9,000 6,000` beside four description rows, or several descriptions in one cell, one per line) — the n-th description is paired with the n-th amount, only when the counts agree.

Documents extracted before structured line items existed fall back to the old heuristic for `subtotal_line_item_consistency` : additional fields whose name suggests a per-line total (name contains `item` , value hint `total` / `amount`), compared with the subtotal only.

Output shape (real example)


```
{
  "result": "flag",
  "details": {
    "date_in_future": {"status": "pass", "reason": "Document date 2026-08-25 is not in the future."},
    "total_tax_consistency": {
      "status": "flag",
      "reason": "Total 197500.00 does not match subtotal 180000.00 + tax 9000.00 (expected 189000.00).",
      "regions": [
        {"field": "amount", "label": "Total amount", "value": "197,500.00 AED",
         "caption": "Field exception: Total amount (197,500.00 AED vs expected 189,000.00)",
         "bounding_box": {"page": 1, "x": 0.7821, "y": 0.5081, "width": 0.1321, "height": 0.0119}}
      ]
    },
    "reference_number_format": {"status": "pass", "reason": "…"},
    "subtotal_line_item_consistency": {"status": "pass", "reason": "Subtotal matches the sum of 3 line-item
field(s)."}
  }
}
```


Unlike the forensic checks, `details` is a **dict keyed by sub-check**, not a list of findings. Each sub-check is `{status, reason, regions?}` where `status` is `pass`, `flag` or `skipped` (no value to check, or unparseable); `regions` (only on `flag`) are drawn as **purple field exception** boxes over the PDF. If a field's position could not be located, the exception is still reported without a box. Rows stored before locations existed are re-derived at read time (`with_field_regions`) without modifying the stored row.

Known limitations

- Depends entirely on extraction quality; a wrong extracted value produces a wrong flag (or a miss).
- Amounts must agree to the cent (see above). A document that rounds each line differently from its printed total by more than a cent per term will flag.
- Line items and the amount in words come from the extraction model. A line it fails to list, or a number it misreads, produces a miss or a wrong flag; the reason text always shows the numbers used.
- `date_in_future` compares against `date.today()` on the worker's machine, not the case's submission date, so a worker with a wrong clock can mis-flag.

Risk rules fed

`rule_id`	Sub-check	Weight	Severity
`field.date_in_future`	`date_in_future`	25	high
`field.line_item_arithmetic_mismatch`	`line_item_arithmetic`	25	high
`field.subtotal_line_item_mismatch` (v3: totals and installments not summed)	`subtotal_line_item_consistency`	15	medium
`field.installment_sum_mismatch`	`installment_consistency`	15	medium

rule_id	Sub-check	Weight	Severity
field.date_sequence_inconsistent	date_sequence	15	medium
field.fake_scan_watermark	fake_scan_watermark	25	high
signature.synthetic_stamp	stamp_authenticity	25	high
signature.synthetic_stamp_unsigned	synthetic_stamp_unsigned	5	low
field.tax_invoice_without_trn	tax_invoice_trn	15	medium
field.multiple_invoices_same_period	multiple_invoices	10	medium
field.per_invoice_mismatch	per_invoice_checks	15	medium
field.total_tax_mismatch	total_tax_consistency	20	high
field.tax_rate_mismatch (v3: rate read from the printed "VAT (5%)" label when not extracted; a rate above 0 with 0.00 tax on a non-zero base is flagged — no longer "explained" by a 0.00 line)	tax_rate_consistency	15	medium
field.amount_in_words_mismatch	amount_in_words_consistency	25	high
field.reference_number_malformed	reference_number_format	6	low
metadata.document_date_after_file_creation	document_date_vs_file_creation	25	high
signature.stamp_issuer_mismatch (v2: names normalised)	stamp_issuer_consistency	15	medium

(Weights are the template defaults; each company's Reviewer L2s can change them for their company — see 11.)